

Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «Санкт-Петербургский политехнический университет Петра
Великого»

Институт компьютерных наук и кибербезопасности

ОТЧЕТ О КУРСОВОЙ РАБОТЕ

по теме: «Архиватор LZ77»

по дисциплине «Структуры данных»

Выполнили: студент гр. 5151003/50001

Гордиенков З. Ю.

студент гр. 5151003/50001

Мотуш В. Р.

студент гр. 5151003/50001

Ходос М. В.

Руководитель: ассистент

Грибков Н. А.

Санкт-Петербург 2026

Оглавление

ВВЕДЕНИЕ	3
ОПИСАНИЕ ПО	4
ИСПОЛЬЗУЕМЫЕ СТРУКТУРЫ ДАННЫХ	6
АЛГОРИТМЫ В ОСНОВЕ ПО	7
ОПИСАНИЕ РЕАЛИЗАЦИИ	8
ТЕСТИРОВАНИЕ ПО	10
ЗАКЛЮЧЕНИЕ	14
ХОД РАБОТЫ	15

ВВЕДЕНИЕ

В ходе выполнения данной курсовой работы была создана программа-архиватор MZ77 под ОС на основе проектов GNU/Linux. Для реализации графического интерфейса применялась графическая библиотека GTK-4.0, для работы с многопоточностью использовалась POSIX-библиотека pthread. Также использовались заголовочные файлы glibc:

- `stdlib.h` — выделение динамической памяти;
- `stdio.h` — форматирование строк;
- `string.h` — работа со строками;
- `limits.h` — макрос максимальной длины пути;
- `stdatomic.h` — типы и функции для безопасной работы с данными, доступными из параллельных потоков;
- `stdbool.h` — определение типа данных `bool`.

Для разработки применялась система сборки autotools (autoconf 2.72, automake 1.17), система контроля версий git 2.47.3, хостинг проектов github.com .

Целью работы было создание простого в использовании архиватора с удовлетворительным коэффициентом сжатия.

ОПИСАНИЕ ПО

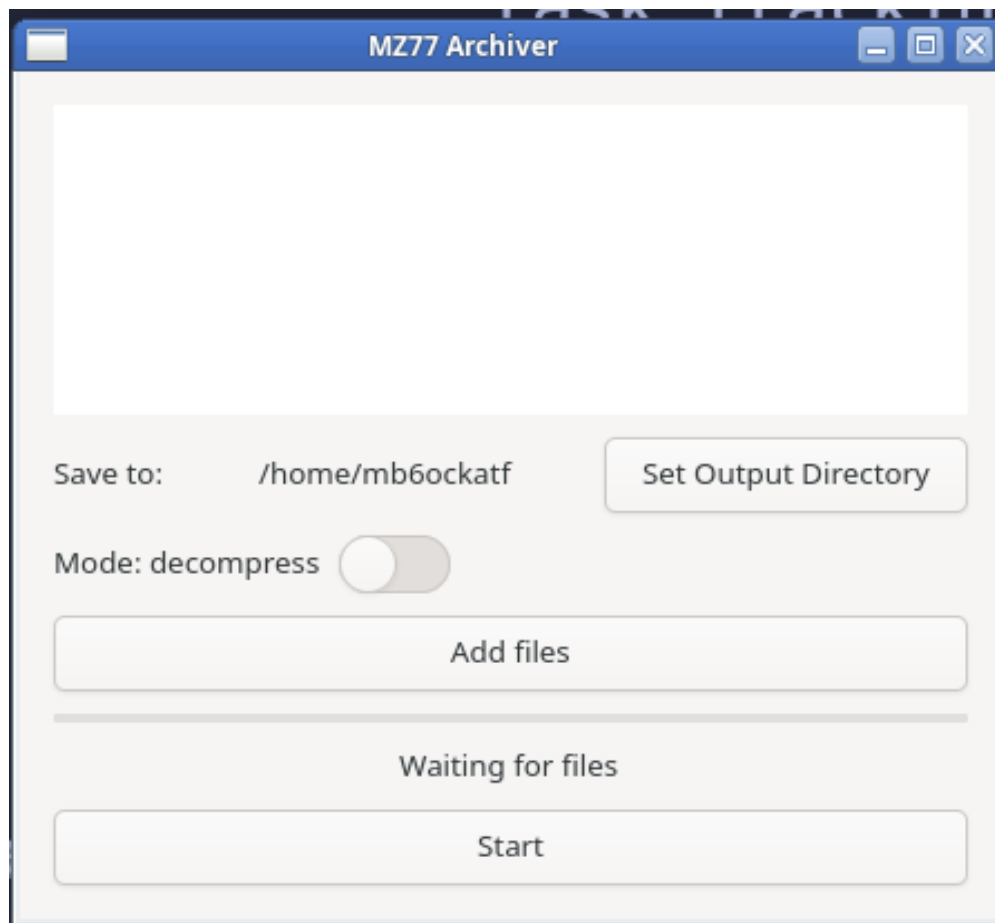


Рис. 0.1 – Окно архиватора

После запуска программы на экране появляется окно graphical tool kit. В нём можно выбрать *исходные файлы*, режим работы (сжатие либо распаковка), целевую директорию – место, куда будут помещены результаты работы.

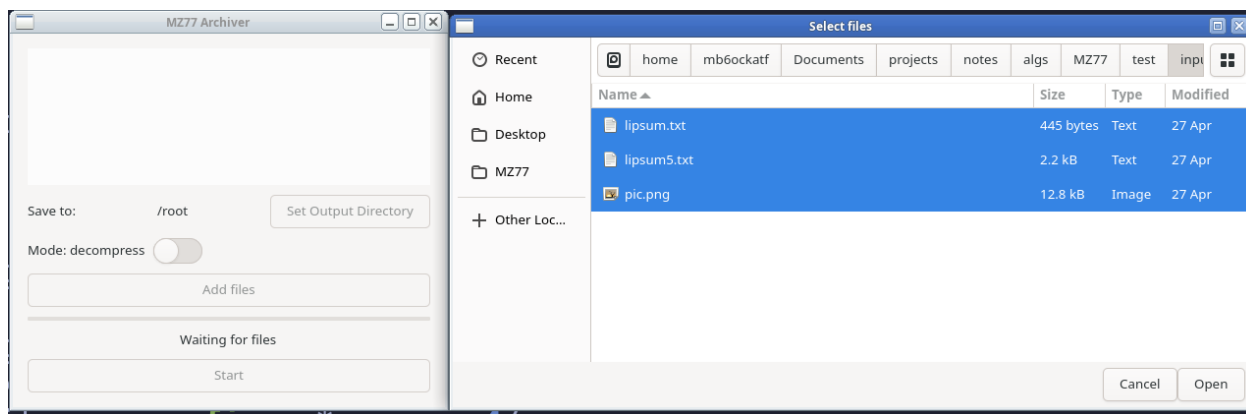


Рис. 0.2 – Окно выбора исходных файлов.

По завершению работы архиватор закрывается автоматически.

ИСПОЛЬЗУЕМЫЕ СТРУКТУРЫ ДАННЫХ

Список выбранных исходных файлов сохраняется в виде очереди. Очередь прикреплена к state-объекту `AppData`, объект очереди — `TaskItem`. Определение очереди доступно в приложении В.

Адреса строк в окне просмотра хранятся в хэш-таблице мощностью 65536. То есть:

$$ht[hash(string)] = address$$

Арифметика адресов позволяет циклически перезаписывать старые записи в хэш-таблице. Определение хэш-таблицы и хэширующая функция доступны в приложении А.

АЛГОРИТМЫ В ОСНОВЕ ПО

В основе работы архиватора лежит алгоритм LZ77. Пусть архиватор находится на некоторой *позиции* в исходном файле. Область 258 байтов после позиции LookAhead, 4Кб до — окно просмотра. Происходит поиск такой максимальной строки, которая уже была когда-то считана из файла и ключ которой содержится в хэш-таблице адресов строк. После нахождения наилучшей строки для замены она заменяется ссылкой на такую строку в окне просмотра. Сама строка уже хеширована, адрес добавлен в таблицу. Старые записи в хэштаблице перезаписываются новыми автоматически из-за циклической индексации.

Замена строки ссылкой на элемент окна просмотра происходит с помощью битового интерфейса ввода-вывода. Вначале записывается бит флага, затем ссылка на окно просмотра, затем длина строки. При закрытии интерфейса происходит выравнивание под целое количество байт. Строки менее 3 байт в LookAhead игнорируются, так как ссылка на них будет занимать больше места, чем сама строка.

Реализован соответствующий битовый интерфейс на чтение. При считывании новых байтов происходит проверка на флаг. В случае если это ссылка, происходит вывод цели ссылки в файл. В случае, если это не ссылка, происходит вывод байта в файл.

Реализация запаковки и распаковки доступна в приложении С.

ОПИСАНИЕ РЕАЛИЗАЦИИ

В программе реализована простая многопоточность. В первом потоке (`g_main_loop_run`) происходит отрисовка графики, его контролирует `graphical tool kit`. Из него по коллбеку `on_start` запускается второй поток (`worker`), в котором происходит работа с файлами.

Из главного потока вызывается `queue_push`, который захватывает мьютекс и устанавливает сигнальную переменную на очереди. Рабочий поток пытается извлечь элемент из очереди, ждёт по сигналу пока очередь станет непустой или пока не будет выставлен флаг `shutdown`.

Общий на два потока объект `AppData` контролирует прогресс-бар, защищает рабочий поток от повторного запуска в случае если он уже запущен, хранит указатели на все элементы окна `graphical tool kit`.

Прямые вызовы функций `graphical tool kit` запрещены за пределом основного цикла. Поэтому они добавляются в цикл основного потока через `g_idle_add`. Главный цикл завершается по `g_main_loop_quit`.

По условию дополнительного задания, были реализованы 3 способа защиты от запуска отладчика:

- `prctl(PR_SET_DUMPABLE, 0)`. Делает невозможным присоединение отладчика для непривилегированного пользователя, запрещает создание дампа памяти процесса. Функция из `sys/prctl.h`;
- Проверка виртуальной файловой системы `proc` на содержание идентификационного номера процесса, который является родительским текущему процессу. Иными словами, обнаруживается случай, когда процесс запускается средством отладки (например, как это делает `gdb: gdb ./program`). Завершение при обнаружении такой ситуации;

- Вызов отладочного прерывания `int3` самостоятельно. При работающем отладчике прерывание `int3` будет обработано отладчиком, затем управление будет передано в программу. Без отладчика, процесс получит соответствующий сигнал и выполнение продолжится. Таким образом, стоит лишь проверить, получен ли сигнал или нет: если получен, то отладчик его не обработал, то есть процесс запущен без отладчика; если не получен, то значит его обработал отладчик — стоит завершить работу.

Весь код, связанный с защитой от отладчика, находится в `src/antidebugger.c`.

ТЕСТИРОВАНИЕ ПО

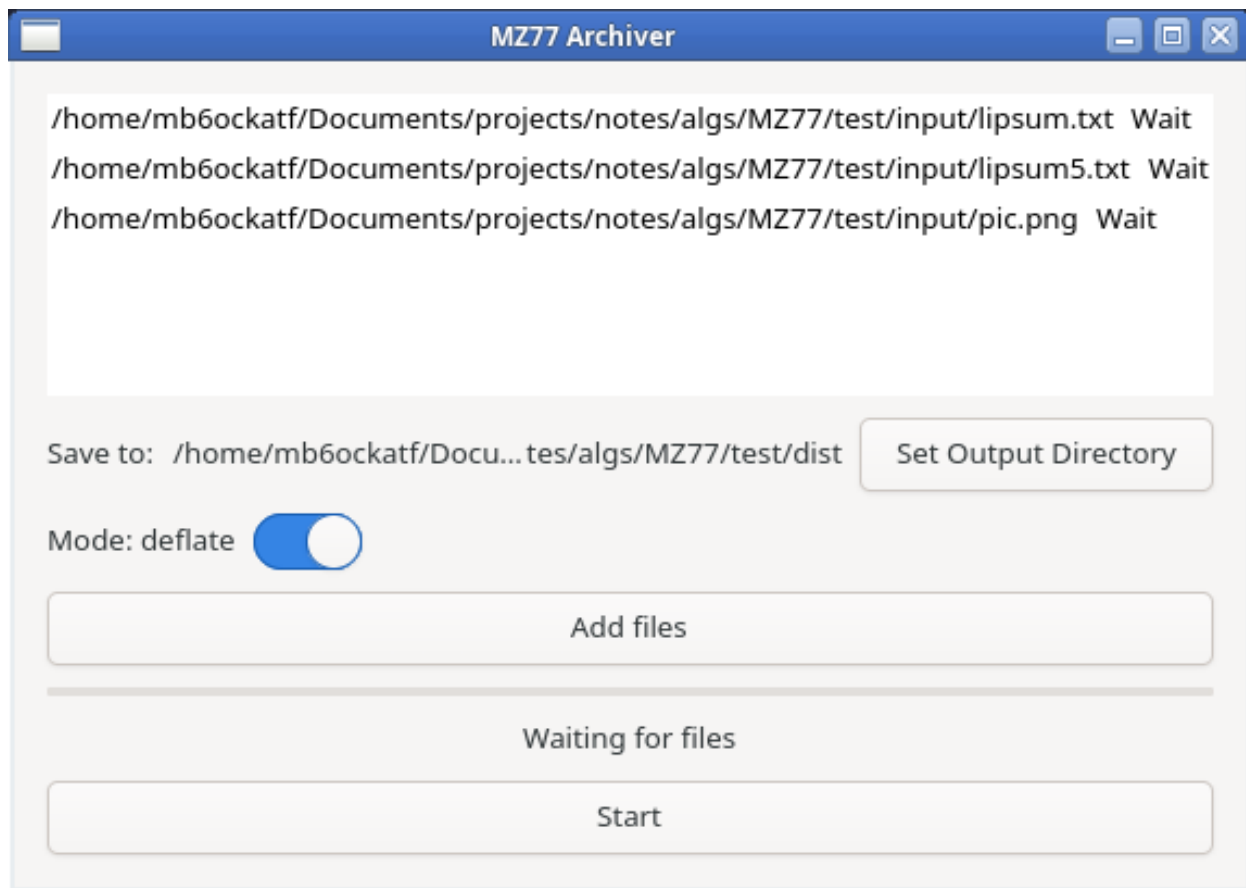


Рис. 0.3 – Тестовый запуск 1

Тестовый запуск архиватора с набором файлов на рис. 0.3.

```
$ ls -la
total 32K
drwxrwxr-x 5 mb6ockatf 4.0K Apr 27 19:31 ../
drwxrwxr-x 2 mb6ockatf 4.0K Apr 27 21:30 ./
-rw-rw-r-- 1 mb6ockatf 2.2K Apr 27 19:28 lipsum5.txt
-rw-rw-r-- 1 mb6ockatf 13K Apr 27 19:27 pic.png
-rw-rw-r-- 1 mb6ockatf 445 Apr 27 19:28 lipsum.txt
$ |
```

Рис. 0.4 – Размеры файлов в первом тесте

Размеры файлов представлены на рис. 0.4.

```
$ ls -la
total 32K
-rw-rw-r-- 1 mb6ockatf 431 May 5 09:59 lipsum.txt.lz77
drwxrwxr-x 5 mb6ockatf 4.0K Apr 27 19:31 ../
-rw-rw-r-- 1 mb6ockatf 14K May 5 09:59 pic.png.lz77
-rw-rw-r-- 1 mb6ockatf 449 May 5 09:59 lipsum5.txt.lz77
drwxrwxr-x 2 mb6ockatf 4.0K Apr 27 21:30 ./
$ |
```

Рис. 0.5 – Размеры результирующих файлов в первом тесте

На рис. 0.5 видно, что один файл был сжат более чем в 4 раза. Изображение почти не было сжато из-за недостаточно большого окна просмотра и малого количества повторяющихся последовательностей байтов.

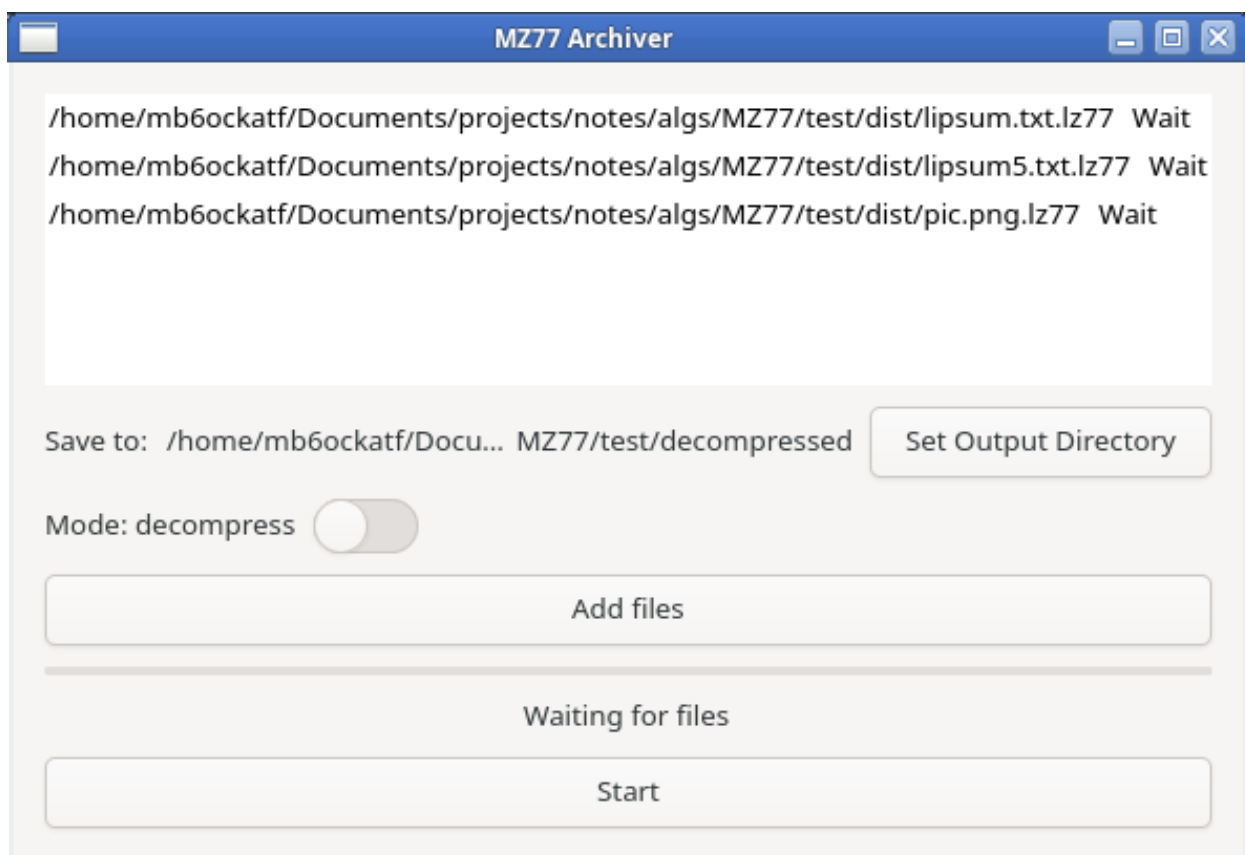


Рис. 0.6 – Тестовый запуск 2

Тестовый запуск архиватора с набором файлов на рис. 0.6. В данном случае — распаковка с целью проверки сохранения целостности файлов.

```
$ ls -la
total 32K
drwxrwxr-x 5 mb6ockatf 4.0K Apr 27 19:31 ../
drwxrwxr-x 2 mb6ockatf 4.0K Apr 27 21:31 ./
-rw-rw-r-- 1 mb6ockatf 2.2K May 5 10:04 lipsum5.txt
-rw-rw-r-- 1 mb6ockatf 13K May 5 10:04 pic.png
-rw-rw-r-- 1 mb6ockatf 445 May 5 10:04 lipsum.txt
$ |
```

Рис. 0.7 – Размеры результирующих файлов во 2-м тесте

На рис. 0.7 видно, что размеры результирующих файлов совпадают с размерами до архивации.

```
$ md5sum lipsum.txt
39d08e040fcdbab6ebc9ad791c50fbac lipsum.txt
$ md5sum ../input/lipsum.txt
39d08e040fcdbab6ebc9ad791c50fbac ../input/lipsum.txt
$ |
```

Рис. 0.8 – Сохранение целостности первого файла

```
$ md5sum lipsum5.txt
c1e4d36211d0e2daec271ec5e19328ca lipsum5.txt
$ md5sum ../input/lipsum5.txt
c1e4d36211d0e2daec271ec5e19328ca ../input/lipsum5.txt
$ |
```

Рис. 0.9 – Сохранение целостности 2-го файла

```
$ md5sum pic.png
93859e0888ee4fd27e554d360589f8cc pic.png
$ md5sum ../input/pic.png
93859e0888ee4fd27e554d360589f8cc ../input/pic.png
$ |
```

Рис. 0.10 – Сохранение целостности 3-го файла

На рис. 0.8, рис. 0.9 и рис. 0.10 видно, что файлы не были повреждены в процессе запаковки и распаковки. Для вычисления контрольной суммы по алгоритму MD5 была использована утилита md5sum из пакета gnu coreutils.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы был получен опыт работы с многопоточностью, создания графических интерфейсов на С с помощью возможностей библиотеки graphical tool kit 4, изучен и реализован алгоритм сжатия LZ77, изучены и реализованы методы защиты программы от запуска отладчика.

Был получен опыт командной работы над программным продуктом с использованием пакета git.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Peter Wright Beginning GTK+/GNOME Programming. - Birmingham: Wrox Press, 2000. - 638 с.

ПРИЛОЖЕНИЕ А

Листинг 1. Определение хэш-таблицы и её методов.

```
typedef struct HashNode {  
    DWORD position;  
    struct HashNode *next;  
} HashNode;  
  
typedef struct {  
    HashNode *buckets[HT_SIZE];  
} LZHashTable;  
  
static inline uint16_t  
hash3 (const uint8_t *p)  
{  
    return ((uint16_t) p[0] << 10) ^ (p[1] << 4) ^ p[2];  
}
```


ПРИЛОЖЕНИЕ В

Листинг 2. Определение очереди.

```
typedef struct {
    enum Mode mode;
    enum TaskStatus status;
    union {
        struct { char in_path[PATH_MAX]; char out_path[PATH_MAX]; } paths;
        struct { size_t original; size_t compressed; double ratio; } stats;
        struct { char msg[256]; } error;
    } data;
} TaskItem;

typedef struct TaskNode {
    TaskItem task;
    struct TaskNode *next;
} TaskNode;

typedef struct {
    TaskNode *head, *tail;
    pthread_mutex_t mutex;
    pthread_cond_t cond;
    atomic_int count;
    atomic_bool shutdown;
} TaskQueue;
```

ПРИЛОЖЕНИЕ С

Листинг 3. Реализация запаковки и распаковки.

```
int
lz77_compress (const char *in_path, const char *out_path)
{
    FILE *fin = fopen (in_path, "rb");
    if (!fin)
        return -1;
    FILE *fout = fopen (out_path, "wb");
    if (!fout)
    {
        fclose (fin);
        return -1;
    }
    BitWriter bw;
    bw_init (&bw, fout);
    uint8_t window[WINDOW_SIZE] = { 0 };
    uint32_t ht_heads[HT_SIZE];
    uint32_t ht_next[WINDOW_SIZE];
    memset (ht_heads, 0xFF, sizeof (ht_heads));
    uint8_t buf[WINDOW_SIZE];
    size_t buf_len = fread (buf, 1, sizeof (buf), fin);
    size_t win_pos = 0;
    while (buf_len > 0)
    {
        size_t best_len = 0;
```

```

    size_t best_dist = 0;
    if (buf_len >= MIN_MATCH)
{
    uint16_t h = hash3 (buf);
    uint32_t pos = ht_heads[h];
    while (pos != 0xFFFFFFFF)
    {
        if (window[pos % WINDOW_SIZE] == buf[0] &&
            window[(pos + 1) % WINDOW_SIZE] == buf[1] &&
            window[(pos + 2) % WINDOW_SIZE] == buf[2])
        {
            size_t len = 3;
            while (len < MAX_MATCH && len < buf_len &&
                window[(pos + len) % WINDOW_SIZE] == buf[len])
                len++;
            if (len > best_len)
            {
                best_len = len;
                best_dist = win_pos - pos;
            }
        }
        pos = ht_next[pos % WINDOW_SIZE];
    }
    if (best_len >= MIN_MATCH)
    {
        bw_write (&bw, 1, 1);
        bw_write (&bw, (uint32_t) (best_dist - 1), 12);
        bw_write (&bw, (uint32_t) (best_len - 3), 8);
    }
}

```

```

for (size_t i = 0; i < best_len; i++)
{
    window[win_pos % WINDOW_SIZE] = buf[i];
    if (i + 2 < best_len)
    {
        uint16_t h = hash3 (&buf[i]);
        ht_next[win_pos % WINDOW_SIZE] = ht_heads[h];
        ht_heads[h] = win_pos;
    }
    win_pos++;
}

memmove (buf, buf + best_len, buf_len - best_len);
buf_len -= best_len;
}

else
{
    bw_write (&bw, 0, 1);
    bw_write (&bw, buf[0], 8);
    window[win_pos % WINDOW_SIZE] = buf[0];
    if (buf_len >= 3)
    {
        uint16_t h = hash3 (buf);
        ht_next[win_pos % WINDOW_SIZE] = ht_heads[h];
        ht_heads[h] = win_pos;
    }
    win_pos++;
    memmove (buf, buf + 1, buf_len - 1);
    buf_len--;
}

```

```

        if (buf_len < sizeof (buf) - 3)
        {
            size_t rd = fread (buf + buf_len, 1, sizeof (buf) - buf_len, fin);
            buf_len += rd;
        }
    }
    bw_finish (&bw);
    fclose (fin);
    fclose (fout);
    return 0;
}

```

```

int
lz77_decompress (const char *in_path, const char *out_path)
{
    FILE *fin = fopen (in_path, "rb");
    if (!fin)
        return -1;
    FILE *fout = fopen (out_path, "wb");
    if (!fout)
    {
        fclose (fin);
        return -1;
    }
    BitReader br;
    br_init (&br, fin);
    uint8_t window[WINDOW_SIZE] = { 0 };
    size_t win_pos = 0;
    while (1)

```

```

{
    int flag = br_read (&br, 1);
    if (flag == -1)
break;
    if (flag == 0)
{
    int lit = br_read (&br, 8);
    if (lit == -1)
        break;
    fputc ((uint8_t) lit, fout);
    window[win_pos % WINDOW_SIZE] = (uint8_t) lit;
    win_pos++;
}
    else
{
    int d_enc = br_read (&br, 12);
    int l_enc = br_read (&br, 8);
    if (d_enc == -1 || l_enc == -1)
        break;
    uint32_t dist = (uint32_t) d_enc + 1;
    uint32_t len = (uint32_t) l_enc + 3;
    uint32_t src = (win_pos + WINDOW_SIZE - dist) % WINDOW_SIZE;
    for (uint32_t i = 0; i < len; i++)
    {
        uint8_t b = window[(src + i) % WINDOW_SIZE];
        fputc (b, fout);
        window[(win_pos + i) % WINDOW_SIZE] = b;
    }
    win_pos += len;
}

```

```
}  
    }  
    fclose (fin);  
    fclose (fout);  
    return 0;  
}
```